

CS633: Parallel Computing

Assignment-1

Code Documentation

Group Number 61

Aayush Kumar (230027)

Aritra Ambudh Dutta (230191)

Cezan Vispi Damania (230310)

Suvid Goel (231065)

Swapnil Garg (231069)

Date: February 14, 2026

1 Code Description

Our implementation simulates a **parallel data processing pipeline** using strictly *point-to-point MPI communication*. The system operates on a linear topology where every process functions in a dual capacity: as a **sender**, it dispatches *rank-dependent pseudo-random data* to downstream neighbors at offsets **D1** and **D2**; as a **receiver**, it processes incoming data using specific mathematical transformations.

For **T** iterations, senders transmit arrays of *M* doubles to their targets. The receiving ranks perform element-wise computations—squaring the values for the *D1* channel and computing natural logarithms for the *D2* channel—before returning the results. Upon receiving, the sender updates its buffers using modulo and multiplication rules to prepare for the next cycle.

The simulation concludes with a **binary-tree reduction** to identify the global maximums. This algorithm efficiently gathers the final results at Rank 0 in $O(\log P)$ steps, strictly adhering to the assignment constraint that forbids `MPI_Reduce` for data values.

1.1 Modular Decomposition

The program execution is divided into four main phases: Initial Setup, declaration and Initialization, The Iterative Core, and Final Aggregation.

Phase 1: Declaration, Setup and Initialization Declaration & Setup:

```
1 int main(int argc, char* argv){
2     int rank, p;
3     int m,d1,d2,t,seed;
4     m=atoi(argv[1]);
5     d1=atoi(argv[2]); d2=atoi(argv[3]);
6     t=atoi(argv[4]); seed=atoi(argv[5]);
7     MPI_Init(&argc, &argv);
8     MPI_Comm_rank(COMM, &rank);
9     MPI_Comm_size(COMM, &p);
10
11     double time = 0;
12     MPI_Status status;
13     double* data_d1 = NULL;
14     double* data_d2 = NULL;
15     double* buf_d1 = NULL, *buf_d2=NULL;
```

The program begins by parsing command-line arguments ($M, D1, D2, T, seed$) and initializing the MPI environment.

- **m**: size of array to be initialised and exchanged at each process.
- **d1** and **d2**: The relative position of the ranks with which the data has to be exchanged.
- **seed**: Argument for `srand()` function; so that we can get reproducible results across different executions of the code by giving the same seed value.
- **buf_d1** and **buf_d2** are the outgoing buffers to ranks `rank + D1` and `rank + D2` respectively on the right.
- **data_d1** and **data_d2** are the incoming buffers from the lower ranks (`rank - D1` and `rank - D2` respectively).

Memory allocation & Initialization:

```

1  if(rank + d1 < p){
2      buf_d1 = (double*)malloc(m*sizeof(double));
3  }
4  if(rank - d1 >= 0){
5      data_d1 = (double*)malloc(m*sizeof(double));
6  }
7  if(rank - d2 >= 0){
8      data_d2 = (double*)malloc(m* sizeof(double));
9  }
10 if(rank+d2<p){
11     buf_d2 = (double*)malloc(m*sizeof(double));
12 }
13 if(rank + d1 < p){
14     generate(seed, buf_d1, m, rank);
15     if(rank + d2 < p){
16         for(int i=0;i<m;i++) buf_d2[i]=buf_d1[i];
17     }
18 }

```

Memory is allocated conditionally to strictly adhere to the topology and minimize overhead:

- **buf_d1** and **buf_d2** (outgoing buffers) are allocated only if `rank + D1` is a valid receiver (i.e. `rank + D1 < P`).
- **data_d1** and **data_d2** (incoming buffers) are allocated only if the `rank - D` is a valid sender for that distance (i.e. `rank - D1 >= 0`).

After this, both `buf_d1` and `buf_d2` are initialized with same random values using `srand()` function.

Initialization (Data Generation):

```

1 void generate(int seed, double* arr, int m, int rank){
2     srand(seed);
3     for(int i=0;i<m;i++){
4         arr[i]=(double)rand()*(rank+1)/10000.0;
5     }
6 }

```

The random number generator is seeded using `srand(seed)`. The primary send buffer (`buf_d1`) is populated using the specified formula:

$$\text{buf_d1}[i] = (\text{double})\text{rand}() \times (\text{rank} + 1) / 10000.0$$

To ensure consistency as required by the problem statement, if a rank is also a valid D2-sender, the contents of `buf_d1` are explicitly copied to `buf_d2`.

Phase 2: The Iterative Core (T iterations)

Inside the main loop, each iteration executes four logical rounds.

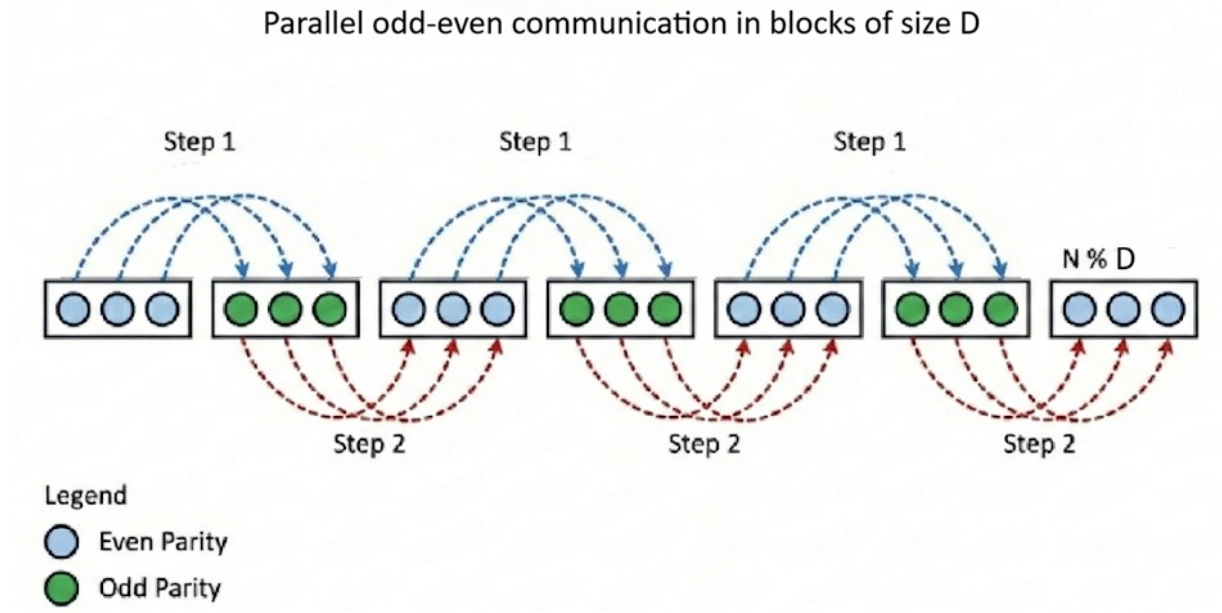
- **Parallel Communication (R1 & R3):** We employed a **block-based parity scheduling** strategy.

Processes are divided into contiguous blocks of size D :

$$\text{parity} = \left\lfloor \frac{\text{rank}}{D} \right\rfloor \bmod 2$$

Any communication pair $(r, r + D)$ always lies in adjacent blocks of opposite parity, thereby enabling parallel communication between the processes.

- **Round 1 (Forward Transfer $R \rightarrow R + D$):**



First, communication occurs between ranks D_1 apart :

$$\text{parity} = \left\lfloor \frac{\text{rank}}{D_1} \right\rfloor \bmod 2$$

- Even blocks: Send $\rightarrow$ Recv
- Odd blocks: Recv $\rightarrow$ Send

```

1 //Communication - Round1
2 if(!((rank/d1)&1)){
3     if(rank + d1 < p)MPI_Send(buf_d1, m, MPI_DOUBLE, rank + d1, 0, COMM);
4     if(rank - d1 >= 0)MPI_Recv(data_d1, m, MPI_DOUBLE, rank - d1, 0, COMM, &status);
5 }
6 else{
7     if(rank - d1 >= 0) MPI_Recv(data_d1, m, MPI_DOUBLE, rank - d1, 0, COMM, &status);
8     if(rank + d1 < p) MPI_Send(buf_d1, m, MPI_DOUBLE, rank + d1, 0, COMM);
9 }

```

Similarly after this ranks D_2 apart communicate, with an updated parity corresponding to D_2 :

$$\text{parity} = \left\lfloor \frac{\text{rank}}{D_2} \right\rfloor \bmod 2$$

```

1 if(!((rank/d2)&1)){
2     if(rank + d2 < p)MPI_Send(buf_d2, m, MPI_DOUBLE, rank + d2, 0, COMM);
3     if(rank - d2 >= 0)MPI_Recv(data_d2, m, MPI_DOUBLE, rank - d2, 0, COMM, &status);
4 }
5 else{
6     if(rank - d2 >= 0) MPI_Recv(data_d2, m, MPI_DOUBLE, rank - d2, 0, COMM, &status);
7     if(rank + d2 < p) MPI_Send(buf_d2, m, MPI_DOUBLE, rank + d2, 0, COMM);
8 }
9

```

Computation (Round 2): After successful reception, valid receiver ranks invoke the dedicated routines `compute_d1()` and `compute_d2()`. These processed results are then returned to the corresponding senders in Round 3.

```

1 //Computation - Round2
2 if(rank-d1 >= 0){
3     compute_d1(data_d1, m);
4     if(rank-d2 >= 0){
5         compute_d2(data_d2, m);
6     }
7 }

```

The function `compute_d1()` performs element-wise squaring on `data_d1`.

```

1 void compute_d1(double* data_d1, int m){
2     for(int i=0;i<m;i++){
3         data_d1[i] =
4             data_d1[i]*data_d1[i];
5     }
6 }

```

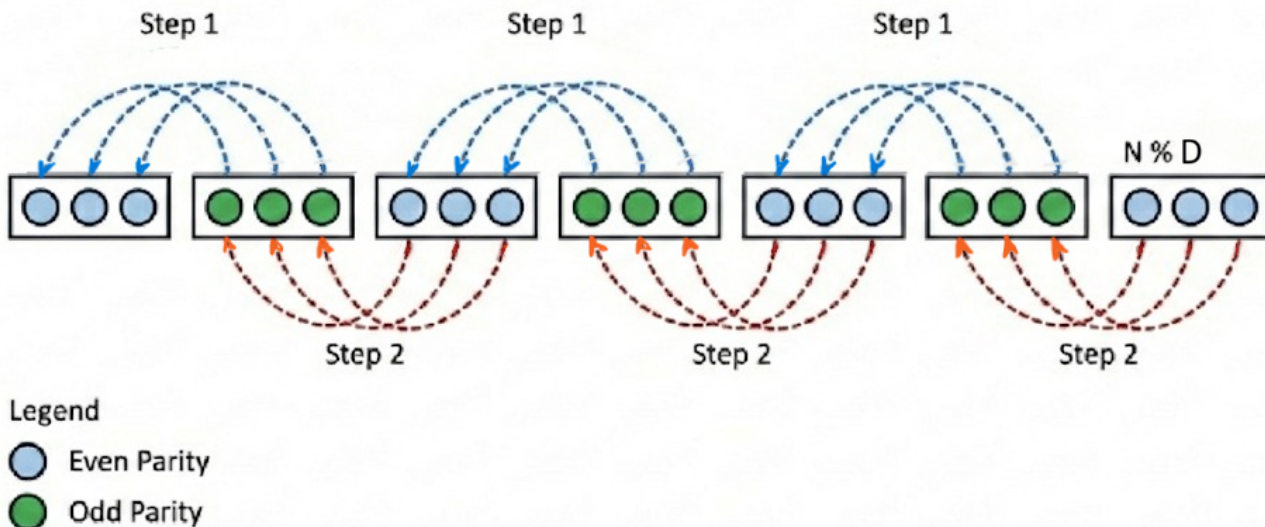
The function `compute_d2()` applies the natural logarithm to `data_d2`.

```

1 void compute_d2(double* data_d2, int m){
2     for(int i=0;i<m;i++){
3         data_d2[i] =
4             log(data_d2[i]);
5     }
6 }

```

Communication : Round 3 (Return Transfer $R + D \rightarrow R$):



- Odd blocks: Send $\rightarrow$ Recv
- Even blocks: Recv $\rightarrow$ Send

```

1 //Communication - Round3
2 if((rank/d1)&1){
3     if(rank - d1 >= 0)MPI_Send(data_d1, m, MPI_DOUBLE, rank - d1, 0, COMM);
4     if(rank + d1 < p)MPI_Recv(buf_d1, m, MPI_DOUBLE, rank + d1, 0, COMM, &status);
5 }
6 else{
7     if(rank + d1 < p) MPI_Recv(buf_d1, m ,MPI_DOUBLE, rank + d1, 0, COMM, &status);
8     if(rank - d1 >= 0) MPI_Send(data_d1, m, MPI_DOUBLE, rank - d1, 0, COMM);
9 }
10 if((rank/d2)&1){
11     if(rank - d2 >= 0)MPI_Send(data_d2, m, MPI_DOUBLE, rank - d2, 0, COMM);
12     if(rank + d2 < p)MPI_Recv(buf_d2, m, MPI_DOUBLE, rank + d2, 0, COMM, &status);
13 }
14 else{
15     if(rank + d2 < p) MPI_Recv(buf_d2, m ,MPI_DOUBLE, rank + d2, 0, COMM, &status);
16     if(rank - d2 >= 0) MPI_Send(data_d2, m, MPI_DOUBLE, rank - d2, 0, COMM);
17 }

```

For every communicating pair, one side posts **Send** first while the other posts **Recv** first. Therefore, each MPI call has a matching counterpart immediately available, enabling parallel communication.

This partitions the ranks into two approximately equal sets ($\approx P/2$ each). For any distance D , all exchanges in a communication phase complete in **two logical steps**.

Update (Round 4):

- **Senders (R4):** After receiving the processed results in Round 3, senders update their buffers using the specified modulo (D_1) or multiplication (D_2) rules.

```

1 if(rank + d2 < p){
2     update_buf_d1(buf_d1, m);
3     update_buf_d2(buf_d2, m);
4 }
5 else if(rank + d1 < p){
6     update_buf_d1(buf_d1, m);
7 }

```

Here, `update_buf_d1` function takes modulo 100000 of data received from `rank + d1`.

```

1 void update_buf_d1(double* arr, int m){
2     for(int i=0;i<m;i++){
3         arr[i] = (ull)arr[i]%100000;
4     }
5 }

```

Here, `update_buf_d2` function multiplies data received from `rank + d2` by `1e+05`.

```

1 void update_buf_d2(double* arr, int m){
2     for(int i=0;i<m;i++){
3         arr[i] = arr[i]*100000.0;
4     }
5 }

```

Phase 3: Final Aggregation

Upon completion of the T iterations, the program transitions to the result aggregation phase. Since the assignment specification prohibits the use of `MPI_Reduce` for data values, we implemented a manual **binary-tree reduction strategy** to consolidate global maximums efficiently.

- **Local Maximum Extraction:** Each valid sender computes two scalar values max1 , max2 from its final buffers. Only these scalars are communicated.
- **Tree Reduction Structure:** The reduction proceeds in stages where the stride $\text{diff} = 1, 2, 4, 8, \dots$ doubles each iteration.
- **Role Determination:** At stride diff , ranks in multiple of $2 \cdot \text{diff}$ receive the data from the ranks having diff remainder on division by $2 \cdot \text{diff}$:

$$\text{Sender if } \text{rank} \bmod (2 \cdot \text{diff}) = \text{diff}, \quad \text{Receiver if } \text{rank} \bmod (2 \cdot \text{diff}) = 0.$$

- **Hierarchical Aggregation:** Senders transmit their partial maxima to rank $r\text{-diff}$ and drop out. Receivers compute:

$$a_{i, \text{new}} = \max(a_{i, \text{local}}, b_i)$$

and continue to the next stage.

- **Convergence:** Since the stride doubles each step, the number of active ranks halves. Thus, global maxima reach Rank 0 in $O(\log P)$ communication steps.

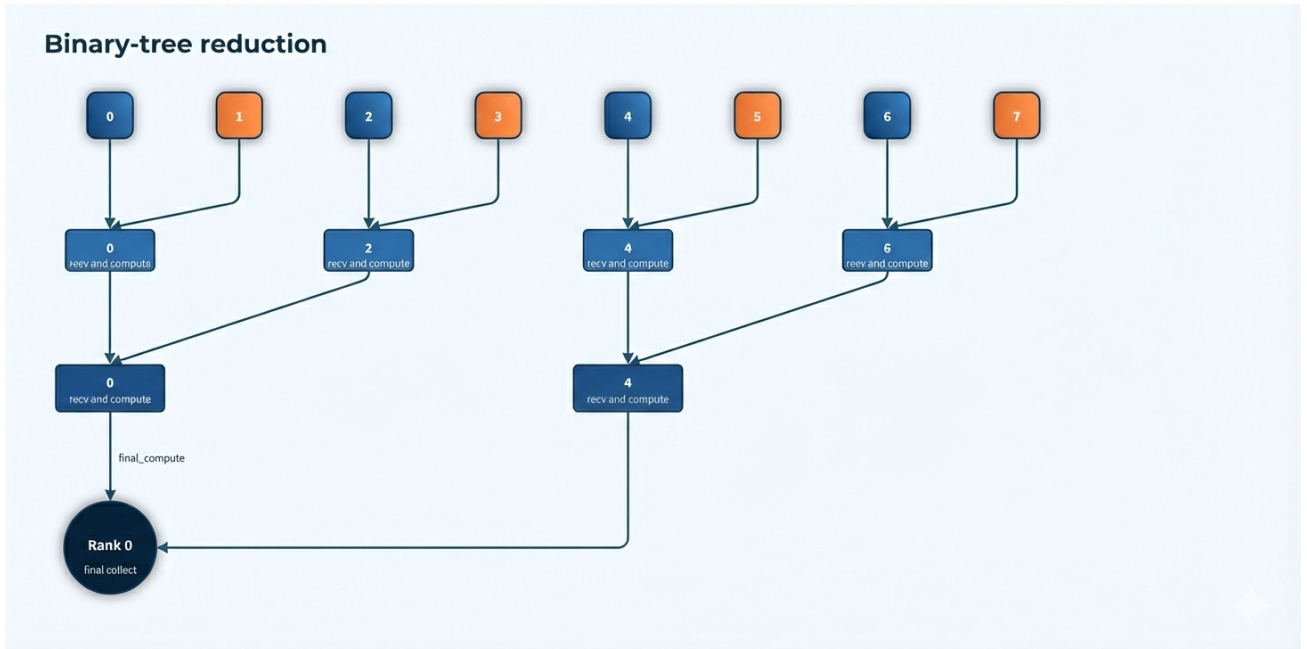


Figure 1: Binary-tree (recursive doubling) reduction pattern used for manual aggregation of local maximum values. Results propagate toward Rank 0 in $O(\log P)$ steps.

Below is the code for aggregation of maximum values from P processes using binary tree reduction:

```

1 //Sending data to rank 0
2 double maxm1=-1, maxm2=-1;
3 double a[2] = {maxm1, maxm2};
4 double time2 = 0;
5 if(buf_d1){
6     if(rank + d1 < p) compute(buf_d1, m, &maxm1);
7     if(rank + d2 < p) compute(buf_d2, m, &maxm2);
8     a[0] = maxm1;
9     a[1] = maxm2;
10    double b[2];
11    int diff = 1;
12    int num = p - d1;
13    while(diff < num){ //binary tree reduction for many to one sends to process 0

```



```

14         if((rank%(diff<<1))==diff){
15             if(rank - diff >= 0)MPI_Send(a, 2, MPI_DOUBLE, rank-diff, 0, COMM);
16         }
17         else if(((rank%(diff<<1))==0) && (rank + diff < num)){
18             MPI_Recv(b, 2, MPI_DOUBLE, rank + diff, 0, COMM, &status);
19             if(a[0] < b[0])a[0]=b[0];
20             if(a[1] < b[1])a[1]=b[1];
21         }
22         diff <= 1;
23     }
24     time2 = MPI_Wtime();
25 }

```

Here, compute function takes the array of m doubles as input from a process and returns the maximum.

```

1 void compute(double* arr, int m, double* maxm){
2     for(int i=0;i<m;i++){
3         if(arr[i] > *maxm) *maxm = arr[i];
4     }
5 }

```

- The variable `diff` represents the stride S and is doubled each iteration using: `diff <= 1;`
- The condition `rank % (diff < 1) == diff` implements the sender rule ($\text{rank} \bmod (2S) = S$).
- The condition `rank % (diff < 1) == 0` implements the receiver rule ($\text{rank} \bmod (2S) = 0$).
- **Hierarchical Aggregation:** Receivers combine incoming partial maxima with their local values using:

$$a[i] = \max(a[i], b[i]),$$

thereby carrying forward aggregated results to the next stride.

- Senders call `MPI_Send(a, 2, ...)` to transmit their partial maxima to rank $r - S$, after which they drop out of subsequent stages. Receivers continue participating until the final result reaches Rank 0.

Global Timing & Reporting: Finally, the total execution time is aggregated using `MPI_Reduce` (with `MPI_Op` as `MPI_MAX`) for calculating Maximum Time among all processes as permitted. Rank 0 then writes the global D1 maximum, D2 maximum, and the elapsed wall-clock time to the output file in the requested format.

```

1     time = time2-time1;
2     double max_time=0;
3
4     MPI_Reduce(&time, &max_time, 1, MPI_DOUBLE, MPI_MAX, 0, COMM); //for getting maximum time
5
6     if(!rank){
7         char filename[32];
8         sprintf(filename, "out_%d.txt", p);
9
10        FILE* fptr = fopen(filename, "a");
11        fprintf(fptr, "%lf %lf %lf\n", a[0], a[1], max_time);
12        fclose(fptr);
13    }

```

Closing the program and freeing the allocated memory:

```

1     if(rank-d1>=0)free(data_d1);
2     if(rank-d2>=0)free(data_d2);
3     if(rank+d1<p)free(buf_d1);
4     if(rank+d2<p)free(buf_d2);
5
6     MPI_Finalize();
7     return 0;

```